

Python notices that the 'cat' string literal assigned to `eggs` is the same as the 'cat' string literal assigned to `spam`; so instead of making a second, redundant string object, it just assigns `eggs` a reference to the same string object that `spam` uses. This explains why the IDs of their strings are the same.

This optimization is called *string interning*, and like the preallocated integers, it's nothing more than a CPython implementation detail. You should never write code that relies on it. Also, this optimization won't catch every possible identical string. Trying to identify every instance in which you can use an optimization often takes up more time than the optimization would save. For example, try creating the 'cat' string from 'c' and 'at' in the interactive shell; you'll notice that CPython creates the final 'cat' string as a new string object rather than reusing the string object made for `spam`:

```
>>> bacon = 'c'
>>> bacon += 'at'
>>> spam is bacon
False
>>> id(spam), id(bacon)
(1285806577904, 1285808207384)
```

String interning is an optimization technique that interpreters and compilers use for many different languages. You'll find further details at https://en.wikipedia.org/wiki/String_interning.

Python's Fake Increment and Decrement Operators

In Python, you can increase the value of a variable by 1 or reduce it by 1 using the augmented assignment operators. The code `spam += 1` and `spam -= 1` increments and decrements the numeric values in `spam` by 1, respectively.

Other languages, such as C++ and JavaScript, have the `++` and `--` operators for incrementing and decrementing. (The name "C++" itself reflects this; it's a tongue-in-cheek joke that indicates it's an enhanced form of the C language.) Code in C++ and JavaScript could have operations like `++spam` or `spam++`. Python wisely doesn't include these operators because